

Guida OOP - Pattern comuni negli esami

Programmazione ad Oggetti - appelli passati e strategia pratica di soluzione

Questo documento raccoglie i pattern ricorrenti emersi dagli appelli passati di OOP. L'obiettivo non e' fare teoria astratta, ma riconoscere rapidamente la forma dell'esercizio e sapere quale struttura di codice usare.

Parte	Pattern dominante	Cosa devi saper fare
Esercizio 1	ADT / modello dati con interfacce, factory, test JUnit	Leggere Test.java, implementare classi Impl, usare collection, Optional, generics e lambda.
Esercizio 2	Piccola GUI Swing a griglia	Implementare la logica separata dalla GUI: stato interno, click, coordinate, fine partita.

1. Pattern comuni dell'Esercizio 1

L'Esercizio 1 cambia tema, ma quasi sempre ti chiede di completare un piccolo modello usando interfacce gia' fornite. Il test JUnit e' l'oracolo: se la consegna e' ambigua, vincono i test.

1.1 Factory + interfaccia + implementazione privata

Pattern: trovi una factory che deve produrre oggetti conformi a una interfaccia. Quasi sempre la soluzione piu' diretta e' creare una classe privata interna che implementa l'interfaccia.

```
public interface Counter {  
    void hit(String key);  
    int count(String key);  
}  
  
public interface CounterFactory {  
    Counter empty();  
}
```

Soluzione tipica:

```
public class CounterFactoryImpl implements CounterFactory {  
  
    @Override  
    public Counter empty() {  
        return new CounterImpl();  
    }  
  
    private static class CounterImpl implements Counter {  
        private final Map<String, Integer> counts = new HashMap<>();  
  
        @Override  
        public void hit(final String key) {  
            counts.put(key, counts.getDefault(key, 0) + 1);  
        }  
  
        @Override  
        public int count(final String key) {  
            return counts.getDefault(key, 0);  
        }  
    }  
}
```

Nota pratica: Quando vedi FactoryImpl, pensa subito: metodo factory -> new Impl(...). Non perdere tempo a cercare strutture elaborate.

1.2 Collections: List, Set, Map

Le collection sono il centro dell'esame. La scelta della struttura dati decide meta' della soluzione.

Situazione	Struttura consigliata	Perche'
Sequenza ordinata di elementi	List<T>	Mantiene ordine e consente accesso per indice.
Elementi senza duplicati	Set<T>	contains e add sono diretti; evita duplicazioni.
Associazione chiave -> valore	Map<K,V>	Serve per conteggi, lookup, raggruppamenti.
Cella di griglia con valore	Map<Position,V>	Ogni posizione diventa una chiave.
Celle marcate	Set<Position>	Ti basta sapere se una cella e' presente.

Esempio: conteggio ricorrenze in una lista.

```
public Map<String, Integer> frequencies(final List<String> words) {  
    final Map<String, Integer> result = new HashMap<>();  
    for (final String word : words) {  
        result.put(word, result.getOrDefault(word, 0) + 1);  
    }  
    return result;  
}
```

Esempio: filtrare una lista senza stream.

```
public List<Integer> onlyPositive(final List<Integer> values) {  
    final List<Integer> result = new ArrayList<>();  
    for (final Integer value : values) {  
        if (value > 0) {  
            result.add(value);  
        }  
    }  
    return result;  
}
```

Nota pratica: In esame non devi dimostrare eleganza. Un ciclo for corretto batte uno stream sbagliato.

1.3 Oggetti immutabili o quasi immutabili

Alcuni esercizi chiedono strutture tipo builder, sequenze o trasformatori. Spesso lo stile migliore e' non modificare l'oggetto corrente, ma restituire un nuovo oggetto con stato aggiornato.

```
public interface ListBuilder<T> {  
    ListBuilder<T> add(T elem);  
    List<T> build();  
}
```

Soluzione immutabile:

```
public class ListBuilderImpl<T> implements ListBuilder<T> {  
    private final List<T> elements;  
  
    public ListBuilderImpl() {  
        this.elements = List.of();  
    }  
  
    private ListBuilderImpl(final List<T> elements) {
```

```

        this.elements = elements;
    }

    @Override
    public ListBuilder<T> add(final T elem) {
        final List<T> copy = new ArrayList<>(this.elements);
        copy.add(elem);
        return new ListBuilderImpl<>(copy);
    }

    @Override
    public List<T> build() {
        return List.copyOf(this.elements);
    }
}

```

Nota pratica: Se i test fanno cose tipo `b1.add(x)` e poi controllano che `b1` non sia cambiato, serve una soluzione immutabile.

1.4 Optional per risultati assenti

Quando un metodo puo' non trovare un risultato, spesso restituisce `Optional<T>`. Non usare `null` se l'interfaccia chiede `Optional`.

```

public Optional<String> firstLongerThan(
    final List<String> words,
    final int minLength) {
    for (final String word : words) {
        if (word.length() > minLength) {
            return Optional.of(word);
        }
    }
    return Optional.empty();
}

```

Uso tipico con `map/filter`:

```

public Optional<Integer> parsePositive(final String text) {
    try {
        final int value = Integer.parseInt(text);
        return value > 0 ? Optional.of(value) : Optional.empty();
    } catch (NumberFormatException e) {
        return Optional.empty();
    }
}

```

1.5 Generics: non evitarli

Molti esercizi sono generici: non lavorano solo con `Integer` o `String`, ma con qualunque `T`. Il trucco e' non fare ipotesi sul tipo, a meno che il test o la firma non lo permetta.

```

public class Box<T> {
    private final T value;

    public Box(final T value) {
        this.value = value;
    }

    public T get() {
        return this.value;
    }
}

```

Esempio con due tipi:

```

public record Pair<X, Y>(X first, Y second) { }

```

```

public <X, Y> List<Pair<X, Y>> zip(
    final List<X> xs,
    final List<Y> ys) {
    final List<Pair<X, Y>> result = new ArrayList<>();
    final int size = Math.min(xs.size(), ys.size());
    for (int i = 0; i < size; i++) {
        result.add(new Pair<>(xs.get(i), ys.get(i)));
    }
    return result;
}

```

1.6 Predicate, Function, BiFunction

Alcuni esercizi chiedono di costruire oggetti che applicano regole. Di solito bastano le functional interfaces standard.

```

public <T> List<T> select(
    final List<T> values,
    final Predicate<T> predicate) {
    final List<T> result = new ArrayList<>();
    for (final T value : values) {
        if (predicate.test(value)) {
            result.add(value);
        }
    }
    return result;
}

```

Esempio di uso:

```

final List<Integer> positives = select(
    List.of(-2, 3, 0, 5),
    x -> x > 0
);

```

1.7 Stream API: utile, ma non obbligatoria

Le soluzioni ufficiali spesso usano stream. Devi conoscerli, ma non devi forzarli se un ciclo e' piu' chiaro.

Obiettivo	Stream tipico
Filtrare	<code>list.stream().filter(p).toList()</code>
Trasformare	<code>list.stream().map(f).toList()</code>
Somma numerica	<code>list.stream().mapToInt(f).sum()</code>
Appiattare liste di liste	<code>lists.stream().flatMap(List::stream).toList()</code>
Raccogliere in Set	<code>stream.collect(Collectors.toSet())</code>

```

public int sumPositive(final List<Integer> values) {
    return values.stream()
        .filter(x -> x > 0)
        .mapToInt(x -> x)
        .sum();
}

```

1.8 Pattern sulle sequenze: windowing, zip, replacer, extractor

Una famiglia molto comune riguarda trasformazioni di liste: prendere finestre, combinare due liste, sostituire elementi, estrarre porzioni. Il metodo e' sempre lo stesso: ciclo sugli indici e attenzione ai limiti.

```

public <T> List<List<T>> windows(final List<T> list, final int size) {
    final List<List<T>> result = new ArrayList<>();
    for (int i = 0; i + size <= list.size(); i++) {
        result.add(List.copyOf(list.subList(i, i + size)));
    }
}

```

```

    }
    return result;
}

// windows(List.of(1,2,3,4), 2) -> [ [1,2], [2,3], [3,4] ]
public <T> List<T> replace(
    final List<T> list,
    final Predicate<T> toReplace,
    final T replacement) {
    final List<T> result = new ArrayList<>();
    for (final T elem : list) {
        result.add(toReplace.test(elem) ? replacement : elem);
    }
    return result;
}

```

2. Pattern comuni dell'Esercizio 2

L'Esercizio 2 usa spesso Swing, ma la parte importante e' la logica. La GUI e' un involucro: tu devi modellare stato, click e condizioni di fine.

2.1 Griglia + Position

Quasi sempre la griglia viene rappresentata con coordinate. Devi avere subito pronte funzioni per controllare se una posizione e' valida.

```

public record Position(int x, int y) { }

private boolean inGrid(final Position p) {
    return p.x() >= 0 && p.x() < size
        && p.y() >= 0 && p.y() < size;
}

```

2.2 Stato interno: Set, Map, Optional

La logica di gioco richiede uno stato. Le strutture ricorrenti sono queste:

Stato da rappresentare	Struttura
Celle cliccate o occupate	Set<Position>
Valore associato a una cella	Map<Position, Integer> o Map<Position, String>
Posizione corrente del giocatore	Position
Obiettivo non sempre presente	Optional<Position>
Sequenza di click	List<Position>

2.3 Metodo hit: il cuore della logica

Il metodo hit/click/move quasi sempre fa tre cose: controlla se il click e' valido, aggiorna lo stato, restituisce un esito.

```

public boolean hit(final Position p) {
    if (!inGrid(p) || marked.contains(p) || gameOver) {
        return false;
    }
    marked.add(p);
    if (marked.size() == target) {
        gameOver = true;
    }
    return true;
}

```

2.4 Vicinato, righe, colonne e diagonali

Molti giochi chiedono di controllare se due celle sono vicine, nella stessa riga, nella stessa colonna o in diagonale.

```
private boolean sameRow(final Position a, final Position b) {
    return a.x() == b.x();
}

private boolean sameColumn(final Position a, final Position b) {
    return a.y() == b.y();
}

private boolean diagonal(final Position a, final Position b) {
    return Math.abs(a.x() - b.x()) == Math.abs(a.y() - b.y());
}

private boolean adjacent(final Position a, final Position b) {
    return Math.abs(a.x() - b.x()) <= 1
        && Math.abs(a.y() - b.y()) <= 1
        && !a.equals(b);
}
```

2.5 Esempio completo di LogicImpl a griglia

Esempio minimale: ogni click marca una cella; il gioco termina quando sono state marcate target celle.

```
public interface Logic {
    boolean hit(Position p);
    boolean isMarked(Position p);
    boolean isOver();
}

public class LogicImpl implements Logic {
    private final int size;
    private final int target;
    private final Set<Position> marked = new HashSet<>();
    private boolean over = false;

    public LogicImpl(final int size, final int target) {
        this.size = size;
        this.target = target;
    }

    @Override
    public boolean hit(final Position p) {
        if (!inGrid(p) || marked.contains(p) || over) {
            return false;
        }
        marked.add(p);
        over = marked.size() >= target;
        return true;
    }

    @Override
    public boolean isMarked(final Position p) {
        return marked.contains(p);
    }

    @Override
    public boolean isOver() {
        return over;
    }

    private boolean inGrid(final Position p) {
```

```

        return p.x() >= 0 && p.x() < size
            && p.y() >= 0 && p.y() < size;
    }
}

```

3. Esempio E1 completo: RulesEngine

Questo esempio simula un esercizio tipico: creare un motore di regole che riceve valori e restituisce le etichette delle regole soddisfatte.

```

public interface RulesEngine<T> {
    void addRule(String name, Predicate<T> rule);
    List<String> apply(T value);
}

public class RulesEngineImpl<T> implements RulesEngine<T> {
    private final Map<String, Predicate<T>> rules = new LinkedHashMap<>();

    @Override
    public void addRule(final String name, final Predicate<T> rule) {
        rules.put(name, rule);
    }

    @Override
    public List<String> apply(final T value) {
        final List<String> result = new ArrayList<>();
        for (final var entry : rules.entrySet()) {
            if (entry.getValue().test(value)) {
                result.add(entry.getKey());
            }
        }
        return result;
    }
}

```

Nota pratica: Qui compaiono insieme Map, generics, Predicate e ordine di inserimento. LinkedHashMap e' utile quando i test si aspettano un ordine preciso.

4. Cosa studiare davvero

Priorita'	Argomento	Perche' serve
1	Collections Java	Sono usate in quasi ogni esercizio.
2	Lettura dei test JUnit	I test spiegano il comportamento atteso meglio della consegna.
3	Factory, interfacce, classi Impl	Struttura base dell'Esercizio 1.
4	Generics	Molte firme sono parametriche: <T>, <X,Y>.
5	Optional	Serve per risultati assenti senza usare null.
6	Lambda e Predicate/Function	Ricorrenti nei modelli basati su regole.
7	Logica su griglie	Cuore dell'Esercizio 2.

5. Strategia operativa in esame

Ordine consigliato:

1. Apri Test.java prima di tutto.
2. Segna quali metodi sono davvero testati.
3. Apri le interfacce e rispetta esattamente le firme.
4. Implementa una soluzione semplice con cicli e collection.
5. Solo dopo, se hai tempo, rendi il codice piu' elegante.
6. Per E2 lavora soprattutto in LogicImpl; tocca la GUI solo se necessario.

Nota pratica: La trappola classica e' partire dalla consegna e immaginare un sistema enorme. No: parti dai test e implementa il minimo comportamento corretto.

6. Template rapidi da ricordare

Ricerca del primo elemento:

```
for (final T elem : list) {  
    if (condition(elem)) {  
        return Optional.of(elem);  
    }  
}  
return Optional.empty();
```

Conteggio con Map:

```
map.put(key, map.getDefault(key, 0) + 1);
```

Copia difensiva:

```
this.list = List.copyOf(inputList);
```

Aggiornamento stato griglia:

```
if (inGrid(p) && !marked.contains(p)) {  
    marked.add(p);  
}
```

7. Sintesi finale

Il pattern generale degli appelli e' stabile: nel primo esercizio devi completare un piccolo ADT con factory, classi Impl e collection; nel secondo devi gestire una logica a griglia separata dalla GUI. Per passare agevolmente non serve conoscere tutto Java: serve essere molto rapido su List, Set, Map, Optional, generics, Predicate/Function e coordinate.